Faculty of Information Technology and Artificial Intelligence
Department of Computer Science
Natural Language Processing — Mini Project 1 (Phase 1)

What Is This Article About?

Classifying Wikipedia Articles by Type with Classical NLP
(DBpedia-14, Corpus Option 6)

Prepared by: Dina Al-Deek · Rand Hussein

Supervisor: Dr. Ahmad Fahel

November 2026

1. Introduction and Problem Statement

Wikipedia has millions of articles, and new ones are written every day. Many tools that work with this content need to know what kind of thing an article describes before they can do anything else with it: a search engine may want to show only films, a knowledge base needs to file a page under "company" or "person", and an editor may want new pages tagged automatically. Doing this by hand at the scale of Wikipedia is not realistic.

Our task is therefore a multi-class text classification problem. The input is the short abstract at the top of a Wikipedia article, and the output is one of six types: **Company, Artist, Athlete, Building, Film** or **WrittenWork**. It is an NLP problem because the type is never stated as a label in the text; it has to be inferred from how the article is written. A film abstract talks about who *directed* and *starred* in it, an athlete abstract about the *league* a person *played* in, and a building abstract about when it was *built* and where it is *located*. The difficult cases are articles that mention one type while describing another, such as a film critic (a person) whose abstract is full of the word "film".

Following Units 1–4 of the course, we built a complete classical pipeline: text preprocessing, corpus statistics, N-gram analysis, a trigram language model for next-word prediction, a Bag of Words representation and two classifiers (Multinomial Naive Bayes and Logistic Regression). The selected model, Logistic Regression on words and word pairs, classified **96.73%** of 30,000 unseen test articles correctly.

2. Dataset and Train/Test Separation

We used DBpedia-14 [1], loaded from Hugging Face (`fancyzhx/dbpedia_14`) [2]. It was built from Wikipedia through the DBpedia project and contains article abstracts labelled with 14 non-overlapping classes. The official split has 560,000 training and 70,000 test articles, with 40,000 and 5,000 articles per class respectively. Each record has a title, the abstract (*content*) and a label.

The project document asks for 4–6 classes. We chose six that cover organisations, people, places and creative works, and that include pairs we expected to be confused (Artist and WrittenWork, Company and Building). We used the abstract only and not the title, because the title is just a name ("Bevin Brothers Manufacturing Company") that usually reappears at the start of the abstract, and we wanted the model to depend on language rather than on names.

Split	Articles	Per class	How it was obtained
Train	18,000	3,000	Random sample of the official training split, `random_state=42`
Validation	1,800	300	10% of our training set, stratified, used only to choose settings
Test	30,000	5,000	All official test articles of the six classes, unchanged

Table 1. Data used in the project

Both sets are perfectly balanced, so no class is favoured during training and accuracy is a fair summary metric. The train and test sets were saved to `data/train.csv` and `data/test.csv` before any modelling, with an `orig_idx` column that points back to the original DBpedia-14 record, so the evaluation can be

reproduced exactly. We checked that no abstract appears in both files. The test set was used only once, for the final evaluation: the vocabulary, the classifier settings and the language-model weights were all chosen using the training data alone.

3. Text Preprocessing

All text passes through one function, `preprocess()` in `src/preprocessing.py`. The same function is applied to the training articles, the test articles and any text typed into the demo; if the three were cleaned differently, the classifier would see word forms it never learned. Table 2 lists the steps.

#	Step	Example	Why
1	Lowercase	Film → film	"Film" and "film" should be one feature
2	Strip accents	María → maria	otherwise accented names break into pieces
3	Remove URLs, digits, punctuation	"in 1938." → "in"	years and symbols do not tell the type
4	Tokenize (NLTK `word_tokenize`)	"family-owned bell" → family, owned, bell	split text into units we can count
5	Remove stop words (NLTK list)	is, a, the, of → removed	very frequent in every class, carry no topic
6	Lemmatize (WordNet)	brothers → brother	merge forms of the same word

Table 2. Preprocessing steps in the order they run

We compared lemmatization with the Porter stemmer on the same sentences. Stemming produced non-words such as *compani*, *histor* and *manufactur*, which made the N-gram tables and the learned features hard to read, while the reduction in vocabulary was similar. We therefore kept the lemmatizer. Removing digits has a visible side effect: "19th century" becomes "th century" and "founded in 1990" becomes "founded in"; both show up later in the N-gram tables and the language model.

For the language model we used a lighter version that keeps stop words and word forms and splits the text into sentences. Words like *was*, *by* and *the* are exactly what a next-word predictor has to learn, so removing them would defeat its purpose.

4. Corpus Statistics

Statistic (training set)	Before preprocessing	After preprocessing
Tokens	856,936	529,455
Vocabulary (distinct words)	60,711	56,723
Average tokens per article	47.6	29.4
Words that occur only once	—	53.4% of the vocabulary

Table 3. Corpus statistics

Preprocessing removed 38% of the tokens, almost entirely stop words, but reduced the vocabulary by only 7%, because most distinct words in these abstracts are names of people, places and works. More than half of the vocabulary occurs only once, the long tail described by Zipf's law. For this reason the classifier ignores words that appear in a single document (`min_df=2`). The articles are short, and Building and WrittenWork abstracts are slightly longer than Athlete ones (Figure 1). The most frequent words after preprocessing — *film, born, american, company, directed, published, historic, book, novel* — are already topical, which suggested early on that counting words would work well.

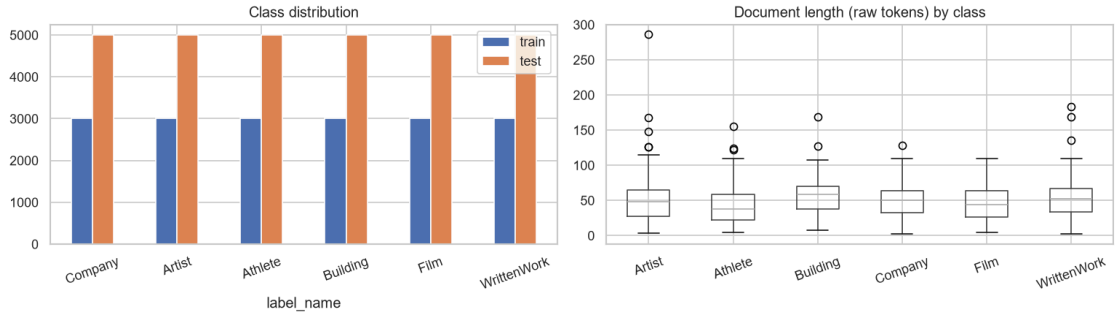


Figure 1. Class distribution (left) and article length by class (right)

5. N-gram Analysis

We counted unigrams, bigrams and trigrams in the training set, overall and per class. To find phrases that *characterise* a class rather than just being frequent, we also ranked bigrams by how often they occur in one class compared with all the others. Table 4 shows a selection; the full lists are saved in `results/`.

Class	Characteristic bigrams	Characteristic trigrams
Company	company headquartered, airline based, holding company	independent record label, wholly owned subsidiary
Artist	singer songwriter, italian painter, stage name	known stage name, american singer songwriter
Athlete	footballer play, football league, summer olympics	major league baseball, professional ice hockey
Building	national register, historic place, parish church	national register historic place, grade ii listed
Film	film directed (1,452), drama film, comedy film	drama film directed, cannes film festival
WrittenWork	first published, peer reviewed, newspaper published	science fiction novel, peer reviewed academic journal

Table 4. Phrases that characterise each class (training set)

Three observations stand out. First, Wikipedia abstracts follow fixed patterns, and multi-word phrases are much more specific than single words: "film directed" appears 1,452 times in Film abstracts, and "national register of historic places" marks a large group of American buildings. Second, some frequent bigrams are shared by almost every class — *united state(s)*, *new york*, *also known* — because they describe places and names, not the type of entity. Third, "born + month" is shared only by Artist and

Athlete, since both describe people; the classifier has to separate them using words like *painter* or *singer* against *footballer* or *league*. These patterns predicted both the high accuracy and the main confusion we later saw between person-like classes.

6. N-gram Language Model

We implemented a bigram and a trigram model (`src/ngram_lm.py`) trained on 43,800 sentences from the training articles, with `<s>` and `</s>` marking sentence boundaries. Probabilities start from counts (maximum likelihood): for example, P(by | directed) = 0.944 and P(in | was founded) = 0.700. Because many word sequences never occur in training, we compared two ways of smoothing on 4,837 sentences from unseen test articles: add-k smoothing (k = 0.01) and linear interpolation of the trigram, bigram and unigram estimates. The interpolation weights were chosen on a 10% slice of the training articles, not on the test set.

Model	Add-k (k = 0.01)	Interpolation	Weights chosen on validation
Bigram	708.1	287.7	0.6 · bigram + 0.4 · unigram
Trigram	3,934.5	**254.0**	0.2 · trigram + 0.5 · bigram + 0.3 · unigram

Table 5. Perplexity on unseen test sentences (lower is better)

With add-k smoothing the trigram model was far worse than the bigram model. Most two-word contexts are rare, so adding k to every one of the ~60,000 possible next words gives unseen words too much probability. With interpolation the trigram became the best model: it uses the longer context when it has evidence for it and falls back on bigram and unigram estimates when it does not. The small trigram weight (0.2) chosen on validation shows how sparse three-word statistics are with under a million training tokens.

Prompt	Top predictions (probability)	Greedy continuation
the film was directed	by (0.94), and (0.04)	… by john ford
he is a professional	baseball (0.14), footballer (0.12)	… baseball player
the company was founded	in (0.70), by (0.19)	… in by the american writer of the same
the novel was written	by (0.62), and (0.13)	… by the american writer of the same name
the building is located	in (0.45), at (0.23), on (0.18)	… in the united states
the airline is based	on (0.72), in (0.22)	… on the national register of historic places in

Table 6. Next-word predictions of the interpolated trigram model (8 prompts in the notebook)

The predictions reflect the house style of Wikipedia, and several of them reveal the type of article ("he is a professional" → baseball, footballer). The continuations also show the limits of a model that sees only two previous words: after a few steps it drifts into generic phrases, and "founded in by" is a direct consequence of removing numbers during preprocessing.

7. Bag of Words and Model Selection

Each article is turned into a vector of word counts with `CountVectorizer`, fitted on the training texts only. We compared two feature sets — single words, and words plus word pairs (bigrams), which carries part of the phrase information from Section 5 into the classifier — and two classifiers, each with three settings. Every configuration was trained on 90% of the training set and scored on the 1,800-article validation set.

Model	Features	Setting	Features	Validation accuracy
Logistic Regression (selected)	words + word pairs	C = 1.0	57,288	**96.72%**
Logistic Regression	words + word pairs	C = 0.5	57,288	96.56%
Logistic Regression	words	C = 0.5	20,052	96.50%
Naive Bayes	words + word pairs	alpha = 1.0	57,288	96.33%
Naive Bayes	words	alpha = 1.0	20,052	95.28%
Naive Bayes	words	alpha = 0.1	20,052	93.39%

Table 7. Selected rows of the 12 validation experiments (all in `results/validation_experiments.csv`)

Word pairs helped both classifiers, and Naive Bayes gained the most from them (+1.1 points), which fits the N-gram analysis: phrases such as “film directed” or “football league” are stronger evidence than their single words. Logistic Regression was better than Naive Bayes in every comparable setting. We selected Logistic Regression with words and word pairs (C = 1.0) and retrained it on all 18,000 training articles, giving a sparse matrix of 18,000 × 63,388 features in which only 0.05% of the entries are non-zero.

8. Results on the Test Set

The final model was evaluated once on the 30,000 official test articles. It reached an **accuracy of 96.73%**, with macro-averaged precision, recall and F1-score all equal to 0.967; 982 articles (3.27%) were misclassified. Validation accuracy was 96.72%, almost identical, so the model generalises to unseen articles rather than memorising the training set. For reference, the best Naive Bayes configuration scored 95.55% on the same test set.

Class	Precision	Recall	F1-score
Company	0.953	0.956	0.954
Artist	0.958	0.943	0.950
Athlete	0.992	0.987	0.990
Building	0.976	0.979	0.977
Film	0.981	0.979	0.980
WrittenWork	0.945	0.959	0.952

Table 8. Per-class results on 30,000 test articles (5,000 per class)



Figure 2. Confusion matrix on the test set (rows: true class, columns: predicted class)

Athlete and Film are the easiest classes: words like *footballer*, *league*, *directed* and *starring* are used by almost no other class. Artist has the lowest recall: 145 artists were predicted as WrittenWork, the largest single error in the matrix. The second group of errors is Company ↔ Building (83 and 80 articles). The notebook also lists 15 individual test predictions, and the prediction for every test article is saved in `results/test_predictions.csv`.

9. Error Analysis

The weights of the classifier agree with the N-gram analysis: the words that push most strongly toward each class are *painter, singer, writer* (Artist), *footballer, player, football* (Athlete), *museum, built, church* (Building), *company, manufacturer, founded* (Company), *film, directed, documentary* (Film) and *novel, newspaper, published* (WrittenWork). Reading the misclassified articles showed three recurring causes.

- **The article mentions one type but describes another.** Writers, cartoonists and illustrators are described through their *books* and what they *published*, so they drift toward WrittenWork. The comic book *American Vampire* is described through the *writer* who created it and was predicted as Artist; the *Arden* estate (Building) is described through the *railroad magnate* who owned it and was predicted as Company.
- **Entities that really belong to two types.** Hotels, restaurant chains, shops and factories are both businesses and places, which explains most Company ↔ Building errors.
- **Label noise.** *Wea*, a Native American tribe, is labelled Company in DBpedia. No model can be "right" on such cases.

All three come from the same limitation: Bag of Words counts which words occur but ignores word order and context, so it cannot tell what an article is *about* from what it *mentions*. The model is,

however, aware of its own uncertainty: the average confidence is 0.97 for correct predictions and 0.72 for wrong ones, so low-confidence predictions could be flagged for a human to check.

10. Demo

The function `predict_article_type(text)` applies the same `preprocess()` function and returns the predicted type with its probability. It runs at the end of the notebook, in `demo.py` from the command line, and in a Google Colab notebook that needs no installation. On six descriptions we wrote ourselves (not taken from the corpus), all six were classified correctly with 97.9–99.2% confidence; for example, "Silent Orbit is a 2019 science fiction thriller directed by James Reed …" → Film (98.1%). We also show a hard case in the presentation: "Jonathan Rosenbaum is an American film critic" is predicted as Film, which illustrates the error pattern discussed in Section 9.

11. Conclusion

A classical pipeline built from Units 1–4 classifies Wikipedia abstracts into six types with 96.73% accuracy on 30,000 unseen articles. The N-gram analysis explains why this works — each type has stable, characteristic phrasing — and why adding word pairs helped. The trigram language model showed in practice how much smoothing matters: the same model went from the worst to the best perplexity when add-k was replaced by interpolation. The remaining errors come from articles that mention one type while describing another, from entities that belong to two types, and from a few wrong labels.

In Project 2 we will keep the same data, split and preprocessing function, use this model as the baseline, and add word embeddings and Transformer models such as BERT. Representations that take context into account should help most on the "mentions versus is about" errors.

References

[1] Zhang, X., Zhao, J., and LeCun, Y. (2015). Character-level Convolutional Networks for Text Classification. *Advances in Neural Information Processing Systems (NeurIPS)*.

[2] DBpedia-14 dataset on Hugging Face: `fancyzhx/dbpedia_14`.

[3] Jurafsky, D., and Martin, J. H. *Speech and Language Processing*, 3rd edition draft. Chapters 2–4.

[4] Bird, S., Klein, E., and Loper, E. (2009). *Natural Language Processing with Python* (NLTK). O'Reilly.

[5] Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.

[6] Course material: NLP Units 1–4 and the NLP Practical Companion Manual, Dr. Ahmad Fahel.